



PROJECT DOCUMENTATION

MindMap - Nested Mind-Map Graph Builder for the Browser

Subject: React JS
Project No.: 122
Program: B.Tech CSE (2025-29)
University: ITM Skills University
Semester: Semester II

Submitted By: Sameer Shaikh
Roll Number: 150096725026
Cohort: Jeff Bezos

Submission: Live Deployment | [GitHub Repository](#)

Table of Contents

- 3.1** Abstract
- 3.2** Student Details
- 3.3** Problem Statement
- 3.4** Objectives
- 3.5** Design / Architecture
- 3.6** Technical Implementation
- 3.7** Technical Specifications
- 3.8** Screenshots of Execution
- 3.9** Results and Findings
- 3.10** Limitations
- 3.11** Conclusion

3.1 Abstract

MindMap is a frontend-only React and TypeScript application that lets users build nested mind-maps on an infinite, pannable, zoomable canvas rather than a fixed-size diagram board. Instead of storing a mind-map as a flat list of shapes, the application models it as a directed, non-cyclic graph: every idea is a node with exactly one parent and any number of children, held as an adjacency list keyed by node ID inside a single global Zustand store.

The project is delivered as two halves built on one shared dark-themed design system — the editor workspace itself (infinite canvas, graph nodes, auto-layout, export/import, undo/redo) and a complete public product shell around it, comprising a marketing landing page, authentication screens, a dashboard for managing saved maps, a starter-template gallery, and settings, profile, and help pages. The application runs entirely in the browser: there is no application server for the core mind-mapping feature set, and persistence is handled locally through a combination of `localStorage` (autosave, last-used preferences, the dashboard's map list) and IndexedDB via the Dexie wrapper, which provides multi-map storage well beyond what `localStorage` alone can hold.

3.2 Student Details

Field	Detail
Student Name	Sameer Shaikh
Roll Number	150096725026
Cohort	Jeff Bezos
Program	B.Tech CSE (2025-29)
University	ITM Skills University
Subject	React JS
Semester	Semester II
Project Number	122
Project Title	MindMap - Nested Mind-Map Graph Builder for the Browser
Implementation	React 19 + TypeScript (Vite, Zustand, Dexie/IndexedDB, frontend-only)

3.3 Problem Statement

Most everyday note-taking and diagramming tools fall into one of two camps: linear documents that cannot express branching relationships between ideas, or heavyweight diagramming suites that demand manual layout and offer no lightweight way to capture a quick brainstorm. Developers and students who want to outline a project, study topic, or product roadmap as a tree of connected ideas run into several recurring problems:

Problem	Description
Linear, rigid documents	Bullet lists and word processors force ideas into a single sequence, with no native way to represent a node having multiple children or a branching hierarchy.
Fixed, bounded canvas	Many lightweight diagram tools cap the drawing area to a single screen or page, forcing the user to scroll, resize, or split a large map across multiple boards.
Manual, tedious layout	Arranging nodes by hand becomes unmanageable once a map grows past a handful of branches; overlapping nodes and crossed connectors are common.
No structural overview	It is difficult to see how deep a hierarchy goes or how many branches an idea has without manually counting nodes.
No portable backup	Sharing or backing up a map usually means taking a screenshot or relying on a paid export feature, rather than a re-importable data file.

No offline, multi-document persistence	Many web-based tools require a server account just to keep more than one document, with nothing built in for storing several maps fully offline.
--	--

MindMap addresses each of these problems with a dedicated, purpose-built interface implemented entirely in React, without introducing a server or external database dependency for the core editing experience.

3.4 Objectives

1. Provide an infinite, pannable, zoomable canvas as the editing surface, instead of a fixed-size diagram board.
2. Model the mind-map as an adjacency-list graph — nodes keyed by ID, each tracking a parent reference and a list of child IDs — rather than a fixed visual layout.
3. Support core node operations: add child, add sibling, edit title inline, recolor from a 12-color accent palette, collapse/expand a subtree, and delete a node together with all of its descendants.
4. Render every parent-child relationship as a smooth, depth-color-coded cubic Bezier curve with a layered glow effect, recalculated live as nodes move.
5. Implement an auto-arrange engine that computes subtree sizes recursively and repositions the entire tree into a balanced, non-overlapping layout on demand.
6. Provide full undo/redo history for graph-mutating actions, plus an automatic save every 30 seconds and a manual save shortcut.
7. Support JSON export and import for full portability between sessions and devices, alongside SVG and rasterized PNG export for sharing a static image of a map.
8. Persist multiple independently named maps using IndexedDB (via Dexie), while keeping fast per-session state (autosave, last-used preferences, the dashboard's map list) in `localStorage`.
9. Provide a dashboard that lists and manages all saved maps, and a template gallery that lets a user start a new map from a pre-built starting structure.
10. Package the editor inside a complete public-facing product: a marketing landing page, authentication flow, dashboard, settings/profile/help pages, and a templates gallery, all sharing one consistent dark visual theme.

3.5 Design / Architecture

System Overview

MindMap is a single-page application built with React 19 and TypeScript, bundled with Vite. React Router DOM v7 drives client-side routing across two shared layout shells — a `LandingLayout` for the public marketing pages and an `AppLayout` for the authenticated dashboard, templates, settings, and profile pages — while the Editor itself renders inside its own standalone full-screen layout outside the normal page chrome. Global application state (the active graph, selection, pan/zoom, and undo history) lives in a single Zustand store; persistence is split across two layers, with `localStorage` handling fast per-session data and Dexie/IndexedDB handling durable, multi-map storage.

Application Layout (Editor)

- **Infinite Canvas Stage** (`<InfiniteCanvas />`) — a fixed, full-viewport container with a dot-grid background that captures global pan (space/middle-mouse drag) and zoom (mouse-wheel with focal-point math) gestures.
- **Vector Edge Layer** (`<VectorEdgeLayer />`) — a full-size SVG layer beneath the node elements that draws the curved, depth-color-coded connectors between parent and child nodes, redrawn live as nodes move.
- **Graph Node** (`<MindMapNode />`) — an absolutely-positioned card with an editable title, a 12-color picker, hover quick-actions for spawning a child, a right-click context menu, and a collapse/expand toggle.
- **Workspace Overlay** (`<WorkspaceOverlay />`) — the floating zoom widget, the telemetry HUD (active branch count, max depth, coordinates), and the top toolbar of undo/redo, auto-arrange, clear, export, import, save, and load actions.

Public Site Structure

- **Landing** — marketing homepage with hero section, a live mini mind-map preview, and feature highlights (infinite canvas, auto-arrange, export to SVG).
- **Auth** — Login, Sign Up, and Forgot Password screens behind a lightweight client-side auth context.
- **Dashboard** — an overview of all saved maps with total-map, total-node, and weekly-activity statistics, plus a card grid of recent maps.
- **Templates** — a gallery of pre-built starting structures (Project Plan, Brainstorming, Study Notes, Meeting Notes, SWOT Analysis, Roadmap).
- **Settings / Profile / Help / About** — user preference, account, documentation, and project-information pages.
- **NotFound** — a catch-all 404 page for unmatched routes.

Module Mapping

Module	Responsibility	Implementation
Infinite Canvas Stage	Captures pan/zoom pointer and wheel input; renders the dot-grid background	<code><InfiniteCanvas /></code>
Vector Edge Layer	Draws color-coded, glow-layered Bezier curves between connected nodes	<code><VectorEdgeLayer /></code> (SVG)
Graph Node	Renders an individual node: inline-editable title, color picker, quick-actions, context menu	<code><MindMapNode /></code>
Workspace Overlay	Zoom controls, telemetry HUD, and the undo/redo & export/import toolbar	<code><WorkspaceOverlay /></code>
Graph Store	Central state: nodes, root ID, pan/zoom, selection, history	<code>useGraphStore.ts</code> / <code>actions.ts</code> / <code>selectors.ts</code>
Auto-Layout Engine	Recursive, subtree-size-based tree layout algorithm	<code>layoutEngine.ts</code> , <code>useAutoLayout.ts</code>
Bezier Routing Engine	Computes cubic Bezier path strings and depth-based edge colors	<code>bezierCalculator.ts</code>
Export / Import Engine	Generates JSON / SVG / PNG downloads; validates imported JSON	<code>exportUtils.ts</code> , <code>validation.ts</code>
IndexedDB Layer	Dexie-backed multi-map database with CRUD helpers	<code>db/indexedDB.ts</code>
Auth Context	Lightweight client-side auth state gating protected routes	<code>contexts/AuthContext.tsx</code> , <code>ProtectedRoute.tsx</code>

3.6 Technical Implementation

3.6.1 Adjacency-List Graph State

Rather than storing the mind-map as nested objects, every node is kept flat in a single record keyed by ID, with explicit `parentId` and `childrenIds` references forming the tree. This keeps lookups, moves, and re-parenting $O(1)$ regardless of how deep the tree grows.

```
interface Node {
  id: string;
  parentId: string | null;
  childrenIds: string[];
  x: number;
  y: number;
  title: string;
  color: string;
  isCollapsed: boolean;
  depth: number;
  metadata: { createdAt: Date; updatedAt: Date; tags: string[] };
}

interface GraphState {
  nodes: Record<string, Node>;
  rootNodeId: string | null;
  panX: number;
  panY: number;
  zoomScale: number;
  selectedNodeId: string | null;
  activeBranches: number;
  maxDepth: number;
}
```

3.6.2 Dynamic SVG Bezier Edge Routing

Connector lines are not straight segments; a small utility reads the coordinates of a parent and child node and returns a cubic Bezier path string, rendered on an SVG layer beneath the node cards. Each edge is drawn three times at different widths and opacities to build up a soft neon glow.

```
function calculateBezierPath(x1, y1, x2, y2): string {
  const dx = Math.abs(x2 - x1);
  const cx1 = x1 + Math.max(dx * 0.4, 80);
  const cy1 = y1;
  const cx2 = x2 - Math.max(dx * 0.4, 80);
  const cy2 = y2;
  return `M ${x1} ${y1} C ${cx1} ${cy1}, ${cx2} ${cy2}, ${x2} ${y2}`;
}
```

Edges are color-coded by depth (indigo at the root, fading to pale lavender at deeper levels), so the overall shape of a large map remains readable at a glance.

3.6.3 Auto-Layout Tree Algorithm

Manually arranging a growing mind-map quickly becomes unmanageable, so the Auto-Arrange action runs a client-side layout pass: it recursively counts the size of every subtree, places the root node at the center, lays out children at fixed horizontal intervals, and distributes vertical space across siblings in proportion to how many descendants each one has, with a minimum spacing floor so small branches never collide.

3.6.4 Collapsible Node Visibility

Every node can act as a visibility gate. Collapsing a node hides its entire subtree and shows a child-count badge in its place; a `selectVisibleNodeIds` selector excludes hidden branches from both rendering and edge-path calculation, keeping large maps responsive.

3.6.5 Pan & Zoom Coordinate Space

The canvas uses its own 2D coordinate system, independent of the browser viewport. Dragging the background translates a `panX / panY` offset; scrolling updates `zoomScale` using focal-point math so the canvas

zooms toward the cursor rather than the origin. A "Fit to Screen" action computes the bounding box of every node and adjusts pan and zoom so the whole map is visible at once.

3.6.6 Dual-Layer Persistence

Two complementary storage layers cover different needs. `localStorage` holds fast, per-session data: a quick-save key per open graph, a rolling autosave key, and the JSON array of map metadata shown on the dashboard. Dexie (an IndexedDB wrapper) backs a dedicated `MindMapDB` database with a `graphs` table, used to store and retrieve multiple independently named maps without the size limits of `localStorage`, upserting by name so saving under an existing map's name updates it in place.

```
// IndexedDB schema (Dexie)
interface SavedGraph {
  id?: number;      // auto-incremented
  name: string;     // graph name
  data: string;     // JSON-stringified graph
  createdAt: Date;
  updatedAt: Date;
}
```

3.6.7 Export / Import Pipeline

The active graph can be exported as a re-importable JSON file (the full node map, root ID, and an export timestamp), as a static SVG (the same node/edge layout redrawn as plain SVG markup with no live interactivity), or as a 2x-scaled PNG rasterized client-side from that SVG. Importing accepts either a picked `.json` file or pasted JSON text, and a `validateGraphData()` type guard checks the required fields before the graph is loaded into the store.

3.6.8 Keyboard Shortcut Dispatch

A single global keyboard handler (`useKeyboardShortcuts`) maps common editing actions to shortcuts — undo/redo, add child/sibling, delete, deselect, zoom in/out/reset, and save — so the editor can be driven largely without the mouse.

3.7 Technical Specifications

The following metrics summarize the scope of the implementation:

Metric	Value
Framework	React 19 + TypeScript 6
Build tool	Vite 8
Routing	React Router DOM v7 (9 routes)
State management	Zustand — single global store, 16 mutation actions
Local persistence layers	2 (localStorage + IndexedDB via Dexie)
Export formats supported	3 (JSON, SVG, PNG)
Built-in starter templates	6 (Project Plan, Brainstorming, Study Notes, Meeting Notes, SWOT Analysis, Roadmap)
Node accent colors	12
Keyboard shortcuts	11
Auto-save interval	30 seconds
External runtime dependencies	Zustand, Dexie, Framer Motion, React Icons, file-saver, uuid, date-fns, clsx, React Router DOM

3.8 Screenshots of Execution

The following screenshots demonstrate MindMap's public site and editor workspace running on the development deployment. Each screenshot corresponds to one core page of the product.

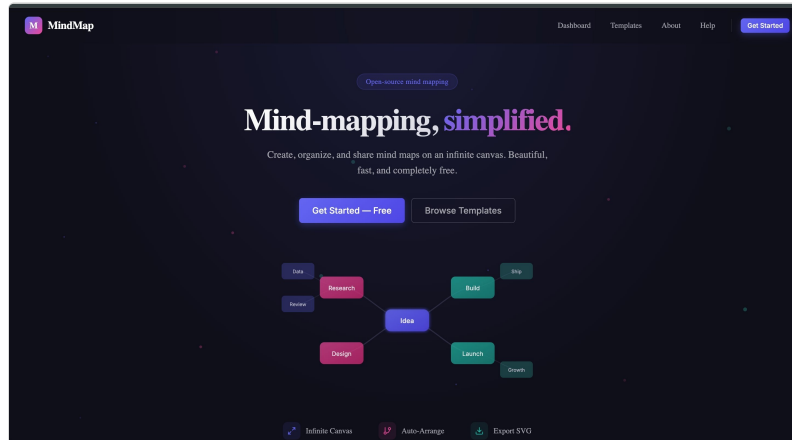


Fig 3.1 — Landing page hero with the "Idea" mind-map preview, infinite canvas, auto-arrange, and export-SVG highlights

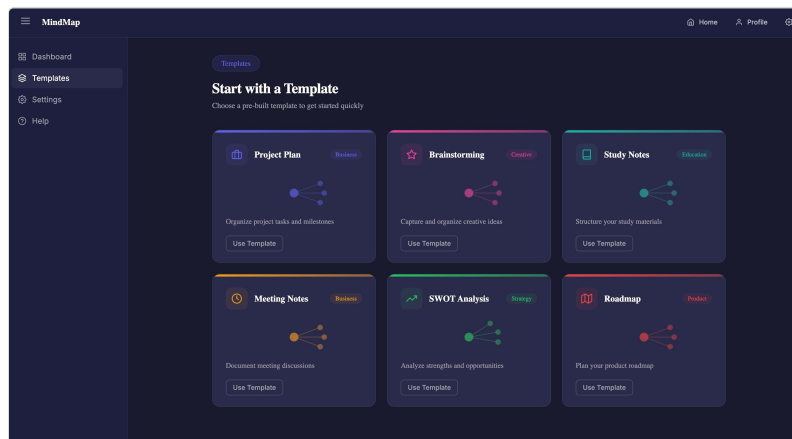


Fig 3.2 — Templates gallery with six pre-built starting structures across business, creative, education, strategy, and product categories

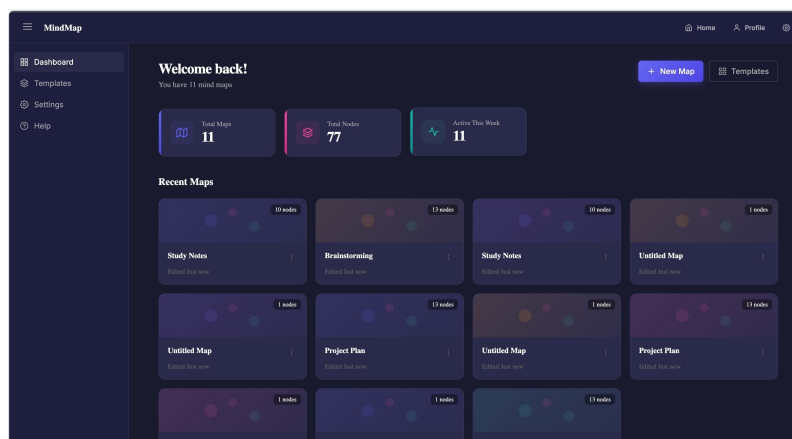


Fig 3.3 — Dashboard showing saved-map statistics and a card grid of recent maps with live node counts

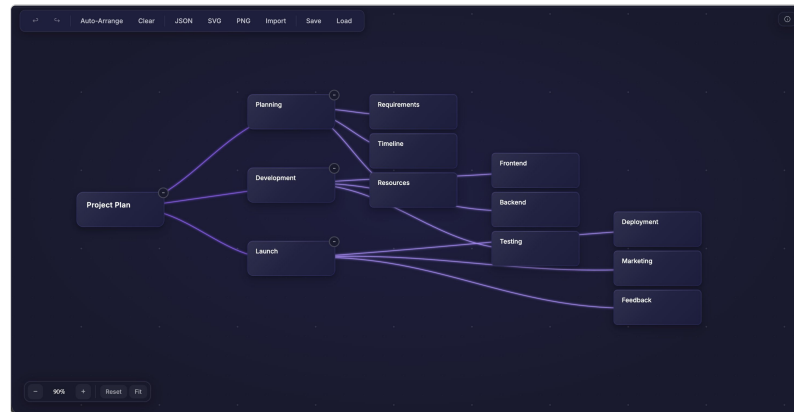


Fig 3.4 — Editor workspace showing a "Project Plan" graph with depth-color-coded Bezier connectors and the zoom/export toolbar

3.9 Results and Findings

Infinite Canvas & Navigation

Panning and zooming behave smoothly across the full canvas, with the zoom widget correctly reporting the current scale (e.g. 90% in the sample editor session) and the Fit action reliably bringing every visible node back into view.

Auto-Arrange Layout

Against the sample "Project Plan" graph — a root node branching into Planning, Development, and Launch, each expanding into further sub-tasks such as Requirements, Resources, Frontend, Backend, Testing, and Deployment — the auto-arrange engine correctly produced a balanced, non-overlapping tree with depth-colored connectors fanning out cleanly from left to right.

Inline Editing & Collapsing

Double-clicking a node title opens it for direct inline editing, with changes reflected immediately in both the node card and the live telemetry stats. Each branch node exposes a collapse toggle that hides its subtree on demand without affecting sibling branches.

Dashboard & Persistence

The dashboard correctly aggregates saved-map statistics across IndexedDB and localStorage — the sample workspace reported 11 total maps, 77 total nodes, and 11 maps active that week — and the recent-maps grid accurately reflected each map's live node count and last-edited status.

Export & Import

The JSON, SVG, and PNG export actions each produced a downloadable file representing the active graph; a previously exported JSON file was successfully re-imported and rendered back onto the canvas with positions, colors, and hierarchy intact.

Templates Gallery

Selecting any of the six starter templates (Project Plan, Brainstorming, Study Notes, Meeting Notes, SWOT Analysis, Roadmap) correctly opened a new editor instance pre-populated with that template's root and first-level branch structure.

3.10 Limitations

- The auto-arrange algorithm uses a simple subtree-proportional spacing rule; very large or unevenly branched trees can still benefit from manual repositioning for ideal readability.
- IndexedDB storage is origin- and browser-scoped, so saved maps do not automatically sync across different browsers or devices; cross-device transfer currently relies on manual JSON export and import.
- Authentication (`AuthContext`) is currently a lightweight, client-side mock that stores a fake user object in `localStorage`; a parallel `authService / apiClient` scaffold exists for a future REST backend but is not yet wired in, so the app should not be treated as having real account security.
- PNG export rasterizes the SVG client-side at 2x resolution in the browser; very large maps may take a moment to render or produce a sizeable file.
- The schema (node) model supports only a single parent per node, so cross-links between unrelated branches — as opposed to strict parent/child relationships — cannot currently be represented.

3.11 Conclusion

MindMap successfully delivers a frontend-only, infinite-canvas alternative to rigid, linear note-taking for capturing branching ideas. The application meets each objective identified in the problem statement: an infinite pannable and zoomable workspace, an adjacency-list graph model with full node CRUD operations, depth-color-coded Bezier connectors, an automatic tree layout engine, undo/redo with autosave, JSON/SVG/PNG export and JSON import, and dual-layer persistence across `localStorage` and IndexedDB — all without a backend server for the core editing experience.

Beyond the editor itself, the project also delivers a complete public product surface — a marketing landing page, authentication flow, a dashboard for managing multiple saved maps, a six-template starter gallery, and settings/profile/help pages — demonstrating both the underlying graph-rendering engineering and a realistic product structure built entirely with React, TypeScript, Vite, Zustand, and Dexie.

Future enhancements could include wiring the existing `authService` / `apiClient` scaffold to a real backend for genuine multi-device accounts, real-time multi-user collaboration on a shared map, touch-gesture support for tablets, and optional cloud sync layered on top of the current local-first storage model.

Submitted by: Sameer Shaikh | Roll: 150096725026 | Cohort: Jeff Bezos
ITM Skills University | B.Tech CSE | Semester II